МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
**«Санкт-Петербургский политехнический
университет Петра Великого»**
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01_03 Технологии машинного обучения и искусственного интеллекта

# Отчёт о выполнении лабораторной работы №6
# Основы архитектуры ЦВМ

**Выполнил:**
Студент группы 5130201/50302 _____ Калинский В.И.
**Принял:**
К.т.н., доцент, доцент ВШТИИ и КОЦ ИИ _____ Вербова Н.М.

«____»_____ 2026 г.

Санкт-Петербург, 2026

**Цель работы:**

Познакомиться с системой процессорных команд и способами использования этих команд. Освоить методику изучения регистровой модели, системы команд и способов адресации с использованием экранного отладчика среды Visual Studio.

**Постановка задачи (вариант 2-8):**

Для группы 50302 в соответствии с методическими указаниями задан вариант 2-8: массив `int iMas[10]`, итератор `unsigned int i`. В цикле `for (i = 0; i < 9; i++)` реализуется вычисление элементов массива: если $i \geq 4$, то $iMas[i] = 13 \cdot i + 7$, иначе $iMas[i] = (i \cdot 8) \oplus 12$. В подготовительной части исследуется прямое и косвенное присваивание констант различной длины: 12, 12345, 12345678, `0x123456789ABCDE` и `0xAB12` через указатель `pI = &iMas[2]`.

**1. Исходная программа и сопоставление с дизассемблером**

В соответствии с шагом 1 методики в программе-шаблоне циклический фрагмент заменён кодом варианта 2-8 (листинг 1).

Листинг 1: Исходная программа (вариант 2-8, глобальные переменные)

```
int iMas[10];
int *pI;
unsigned int i;

int main() {
    iMas[3] = 12;
    iMas[4] = 12345;
    iMas[5] = 12345678;
    iMas[6] = 0x123456789ABCDE;

    pI = &iMas[2];
    *pI = 0xAB12;

    for (i = 0; i < 9; i++) {
        if (i >= 4)
            iMas[i] = 13 * i + 7;
        else
            iMas[i] = (i * 8) ^ 0xC;
    }
    return 0;
}
```

Программа оттранслирована в среде Visual Studio. В режиме отладки (окно Disassembly) получен машинный код (листинг 2) с сопоставлением инструкциям языка Си.

Листинг 2: Дизассемблированный код с сопоставлением инструкциям Си

```
    1: int iMas[10];
2: int *pI;
3: unsigned int i;
4:
5: int main() {
00007FF630CD1720 40 55                 push        rbp
                 ; Сохранение базового регистра кадра
00007FF630CD1722 57                    push        rdi
                 ; Сохранение регистра rdi
00007FF630CD1723 48 81 EC E8 00 00 00  sub         rsp,0E8h
                 ; Выделение памяти в стеке под локальные нужды
00007FF630CD172A 48 8D 6C 24 20        lea         rbp,[rsp+20h]
                 ; Установка базового указателя кадра
00007FF630CD172F 48 8D 0D CA F8 00 00  lea         rcx,[
__B952A550_main@cpp (07FF630CE1000h)] ; Служебный вызов отладчика
00007FF630CD1736 E8 3A FC FF FF        call
__CheckForDebuggerJustMyCode (07FF630CD1375h) ; Проверка режима Just My
Code
00007FF630CD173B 90                    nop
                 ; Выравнивание
 6:  iMas[3] = 12;
00007FF630CD173C B8 04 00 00 00        mov         eax,4
                 ; Размер элемента (4 байта для int)
00007FF630CD1741 48 6B C0 03           imul        rax,rax,3
                 ; Смещение элемента: 3 * 4 = 12 (0x0C) байт
00007FF630CD1745 48 8D 0D F4 AA 00 00  lea         rcx,[iMas (07
FF630CDC240h)]           ; Загрузка адреса массива в rcx
00007FF630CD174C C7 04 01 0C 00 00 00  mov         dword ptr [rcx+rax],0
Ch               ; Базовоиндексная- запись константы 12
 7:  iMas[4] = 12345;
00007FF630CD1753 B8 04 00 00 00        mov         eax,4
                 ; Размер элемента
00007FF630CD1758 48 6B C0 04           imul        rax,rax,4
                 ; Смещение элемента: 4 * 4 = 16 (0x10) байт
00007FF630CD175C 48 8D 0D DD AA 00 00  lea         rcx,[iMas (07
FF630CDC240h)]           ; Базовый адрес массива iMas
00007FF630CD1763 C7 04 01 39 30 00 00  mov         dword ptr [rcx+rax
],3039h             ; Запись байтовой2- константы 12345
 8:  iMas[5] = 12345678;
00007FF630CD176A B8 04 00 00 00        mov         eax,4
                 ; Размер элемента
```

```
00007FF630CD176F 48 6B C0 05          imul        rax,rax,5
                 ; Смещение элемента: 5 * 4 = 20 (0x14) байт
00007FF630CD1773 48 8D 0D C6 AA 00 00 lea         rcx,[iMas (07
FF630CDC240h)]          ; Базовый адрес массива iMas
00007FF630CD177A C7 04 01 4E 61 BC 00 mov         dword ptr [rcx+rax],0
BC614Eh          ; Запись байтовой4- константы 12345678
9:   iMas[6] = 0x123456789ABCDE;
00007FF630CD1781 B8 04 00 00 00       mov         eax,4
                 ; Размер элемента
00007FF630CD1786 48 6B C0 06          imul        rax,rax,6
                 ; Смещение элемента: 6 * 4 = 24 (0x18) байта
00007FF630CD178A 48 8D 0D AF AA 00 00 lea         rcx,[iMas (07
FF630CDC240h)]          ; Базовый адрес массива iMas
00007FF630CD1791 C7 04 01 DE BC 9A 78 mov         dword ptr [rcx+rax
],789ABCDEh         ; Запись младших 32 бит усечение( константы)
10:
11:      pI = &iMas[2];
00007FF630CD1798 B8 04 00 00 00       mov         eax,4
                 ; Размер элемента
00007FF630CD179D 48 6B C0 02          imul        rax,rax,2
                 ; Смещение для индекса 2: 2 * 4 = 8 байт
00007FF630CD17A1 48 8D 0D 98 AA 00 00 lea         rcx,[iMas (07
FF630CDC240h)]          ; Базовый адрес массива iMas
00007FF630CD17A8 48 03 C8             add         rcx,rax
                 ; rcx = адрес элемента iMas[2]
00007FF630CD17AB 48 8B C1             mov         rax,rcx
                 ; Перенос адреса в rax
00007FF630CD17AE 48 89 05 B3 AA 00 00 mov         qword ptr [pI (07
FF630CDC268h)],rax   ; Сохранение адреса в указатель pI (8 байт)
12:      *pI = 0xAB12;
00007FF630CD17B5 48 8B 05 AC AA 00 00 mov         rax,qword ptr [pI (07
FF630CDC268h)]   ; Считывание адреса из указателя pI в rax
00007FF630CD17BC C7 00 12 AB 00 00    mov         dword ptr [rax],0AB12h
                 ; Косвенная запись константы 0xAB12 по адресу в rax
13:
14:      for (i = 0; i < 9; i++) {
    00007FF630CD17C2 C7 05 A4 AA 00 00 00 00 00 00 mov         dword ptr
 [i (07FF630CDC270h)],0 ; Инициализация счетчика цикла: i = 0
    00007FF630CD17CC EB 0E                jmp         main+0BCh (07
FF630CD17DCh)           ; Безусловный переход к проверке условия цикла
    00007FF630CD17CE 8B 05 9C AA 00 00    mov         eax,dword ptr [i
(07FF630CDC270h)]    ; Загрузка значения i в eax
    00007FF630CD17D4 FF C0                inc         eax
                 ; Инкремент счетчика (i++)
    00007FF630CD17D6 89 05 94 AA 00 00    mov         dword ptr [i (07
FF630CDC270h)],eax    ; Сохранение обновленного i
```

```
00007FF630CD17DC 83 3D 8D AA 00 00 09 cmp     dword ptr [i (07FF630CDC270h)],9      ; Сравнение i с 9
00007FF630CD17E3 73 43                jae     main+108h (07FF630CD1828h)            ; Если i >= 9, выход из цикла беззнаковый( переход)
15:          if (i >= 4)
00007FF630CD17E5 83 3D 84 AA 00 00 04 cmp     dword ptr [i (07FF630CDC270h)],4      ; Сравнение i с 4
00007FF630CD17EC 72 1C                jb      main+0EAh (07FF630CD180Ah)            ; Если i < 4, переход в ветку else (jb, CF=1)
16:              iMas[i] = 13 * i + 7;
00007FF630CD17EE 6B 05 7B AA 00 00 0D imul    eax,dword ptr [i (07FF630CDC270h)],0Dh ; eax = i * 13 (0x0D)
00007FF630CD17F5 83 C0 07             add     eax,7
                      ; eax = 13 * i + 7
00007FF630CD17F8 8B 0D 72 AA 00 00    mov     ecx,dword ptr [i (07FF630CDC270h)]    ; Загрузка индекса i в ecx
00007FF630CD17FE 48 8D 15 3B AA 00 00 lea     rdx,[iMas (07FF630CDC240h)]           ; Базовый адрес массива в rdx
00007FF630CD1805 89 04 8A             mov     dword ptr [rdx+rcx*4],eax             ; Запись в массив: База + Индекс * 4 код( 89 04 8A)
00007FF630CD1808 EB 1C                jmp     main+106h (07FF630CD1826h)            ; Обход ветки else
17:          else
18:              iMas[i] = (i * 8) ^ 0xC;
00007FF630CD180A 8B 05 60 AA 00 00    mov     eax,dword ptr [i (07FF630CDC270h)]    ; Загрузка i в eax
00007FF630CD1810 C1 E0 03             shl     eax,3
                      ; Умножение на 8 сдвигом влево на 3 бита
00007FF630CD1813 83 F0 0C             xor     eax,0Ch
                      ; Побитовое исключающее ИЛИ с константой 12
00007FF630CD1816 8B 0D 54 AA 00 00    mov     ecx,dword ptr [i (07FF630CDC270h)]    ; Загрузка индекса i в ecx
00007FF630CD181C 48 8D 15 1D AA 00 00 lea     rdx,[iMas (07FF630CDC240h)]           ; Базовый адрес массива в rdx
00007FF630CD1823 89 04 8A             mov     dword ptr [rdx+rcx*4],eax             ; Запись в массив: База + Индекс * 4 код( 89 04 8A)
19:          }
00007FF630CD1826 EB A6                jmp     main+0AEh (07FF630CD17CEh)            ; Переход к следующей итерации на( шаг цикла)
20:
21:          return 0;
00007FF630CD1828 33 C0                xor     eax,eax
                      ; Код возврата функции 0 (return 0)
22: }
00007FF630CD182A 48 8D A5 C8 00 00 00 lea     rsp,[rbp+0C8h]
                  ; Восстановление указателя стека
```

```
00007FF630CD1831 5F                    pop         rdi
                 ; Восстановление регистра rdi
00007FF630CD1832 5D                    pop         rbp
                 ; Восстановление базового указателя кадра rbp
00007FF630CD1833 C3                    ret
                 ; Возврат из функции
```

## 2. Список переменных и карта памяти

Переменные `iMas`, `pI`, `i` объявлены вне функций (глобально) и размещены компилятором в статической памяти данных. Их адреса постоянны и зафиксированы отладчиком (таблица 1).

Таблица 1: Список переменных и карта памяти программы

| Имя | Класс | Тип | Размер | Адрес в памяти |
|---|---|---|---|---|
| `iMas[0..9]` | глобальная | `int[10]` | 40 байт | `0x07FF630CDC240` – `0x07FF630CDC267` |
| `pI` | глобальная | `int*` | 8 байт | `0x07FF630CDC268` – `0x07FF630CDC26F` |
| `i` | глобальная | `unsigned int` | 4 байта | `0x07FF630CDC270` – `0x07FF630CDC273` |

Переменные расположены последовательно: массив `iMas` занимает 40 байт ($28_{16}$), следом 8-байтовый указатель `pI`, затем 4-байтовый счетчик `i`.

## 3. Анализ команд обращения к переменным

В таблице 2 систематизированы все команды, обращающиеся к переменным программы, с указанием числа операндов и способов их адресации, а в таблице 3 – 10 команд байтовых в ассемблере с расшифровкой.

Таблица 2: Команды обращения к переменным программы

| **Команда ассемблера** | **Перем.** | **Оп.** | **Операнд 1 (приёмник)** | **Операнд 2 (источник)** |
|---|---|---|---|---|
| `mov [rcx+rax], 0Ch` | `iMas[3]` | 2 | Базово-индексная | Непосредств. |
| `mov [rcx+rax], 3039h` | `iMas[4]` | 2 | Базово-индексная | Непосредств. |
| `mov [rcx+rax], 0BC614Eh` | `iMas[5]` | 2 | Базово-индексная | Непосредств. |
| `mov [rcx+rax], 789ABCDEh` | `iMas[6]` | 2 | Базово-индексная | Непосредств. |
| `mov qword ptr [pI], rax` | `pI` | 2 | Прямая | Регистровая |
| `mov rax, qword ptr [pI]` | `pI` | 2 | Регистровая | Прямая |
| `mov dword ptr [rax], 0AB12h` | `iMas[2]` | 2 | Косвенно-регистровая | Непосредств. |
| `mov dword ptr [i], 0` | `i` | 2 | Прямая | Непосредств. |
| `mov eax, dword ptr [i]` | `i` | 2 | Регистровая | Прямая |
| `mov dword ptr [i], eax` | `i` | 2 | Прямая | Регистровая |
| `cmp dword ptr [i], 9` | `i` | 2 | Прямая | Непосредств. |
| `cmp dword ptr [i], 4` | `i` | 2 | Прямая | Непосредств. |
| `imul eax, dword ptr [i], 0Dh` | `i` | 3 | Регистровая | Прямая и непоср. |
| `mov ecx, dword ptr [i]` | `i` | 2 | Регистровая | Прямая |
| `mov [rdx+rcx*4], eax` | `iMas[i]` | 2 | Масштабно-индексная | Регистровая |

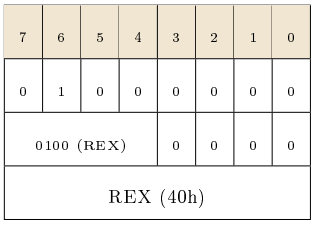
**4. Рассмотрим каждую ассемблерную команду подробно (процессор x86)**

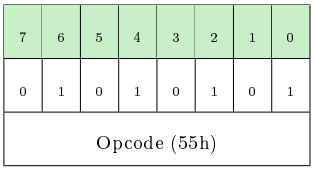
Для 10 команд программы, систематизированных в таблице 3, ниже представлен подробный побайтовый и побитовый разбор структуры машинного кода в соответствии с общим форматом команды архитектуры x86 (префиксы, код операции, байт ModR/M, байт SIB, поле смещения Displacement и непосредственный операнд Immediate).

Таблица 3: Примеры машинных команд и их байтового представления

| **Байт** | **Ассемблер** | **Пояснение** |
|---|---|---|
| 40 55 | push rbp | Сохранение регистра rbp |
| 48 81 EC E8 00 00 00 | sub rsp, 0E8h | Выделение стека |
| B8 04 00 00 00 | mov eax, 4 | Загрузка 4 в eax |
| 48 6B C0 03 | imul rax, rax, 3 | Умножение rax на 3 |
| 48 8D 6C 24 20 | lea rbp,[rsp+20h] | Загрузка адреса в rbp |
| FF C0 | inc eax | Инкремент eax |
| 83 3D 8D AA 00 00 09 | cmp dword ptr [i], 9 | Сравнение i с 9 |
| 73 43 | jae main+108h (07FF630CD1828hP) | Беззнаковый переход |
| 33 C0 | xor eax, eax | Обнуление eax |
| C3 | ret | Возврат из функции |

**00007FF630CD1720** **40 55** `push rbp`



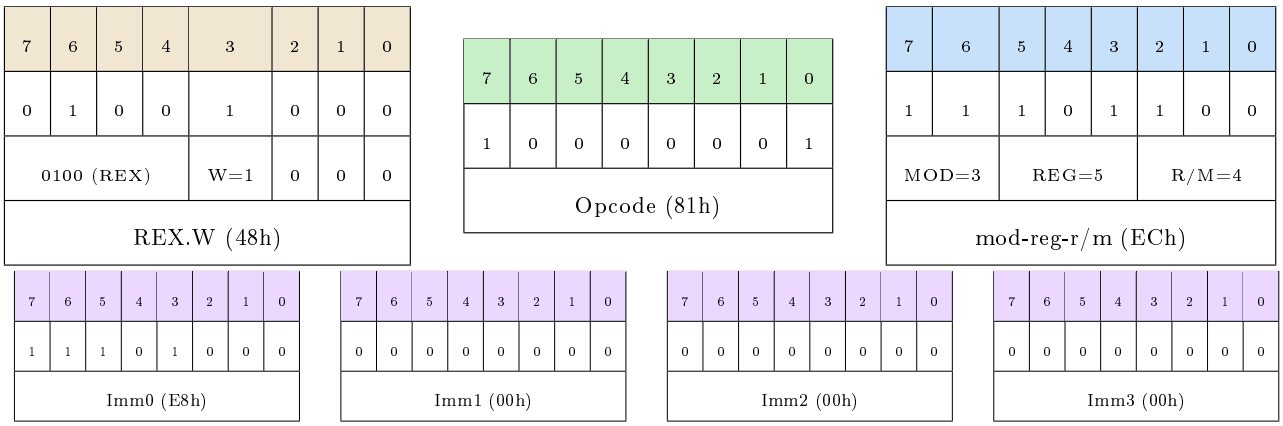




`5xh - push`

`x5h - rbp`

Неявная адресация

**00007FF630CD1723** **48 81 EC E8 00 00 00** `sub rsp, 0E8h`




81 - SUB

MOD = 3 - регистровая адресация (R/M содержит номер регистра)

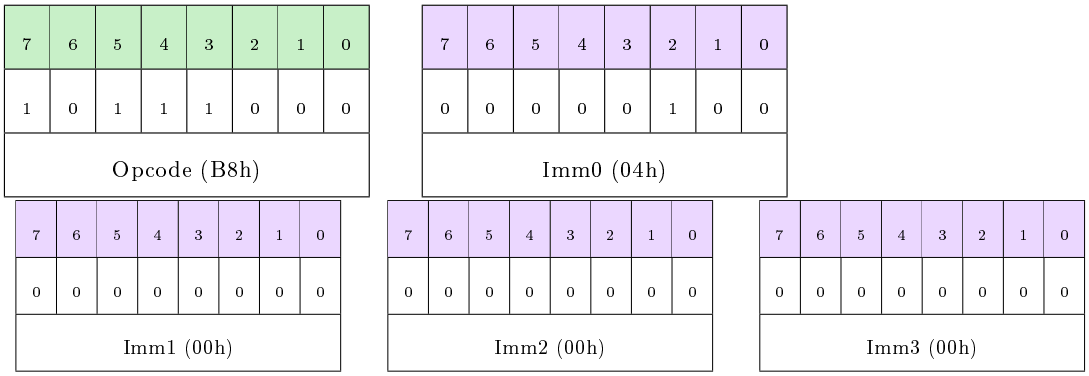
REG = 5 код операции SUB

R/M = 4 - rsp

Immediate = 000000E8h (выделение 232 байт в стековом кадре)

Непосредственная адресация

**00007FF630CD173C** **B8 04 00 00 00** `mov eax, 4`



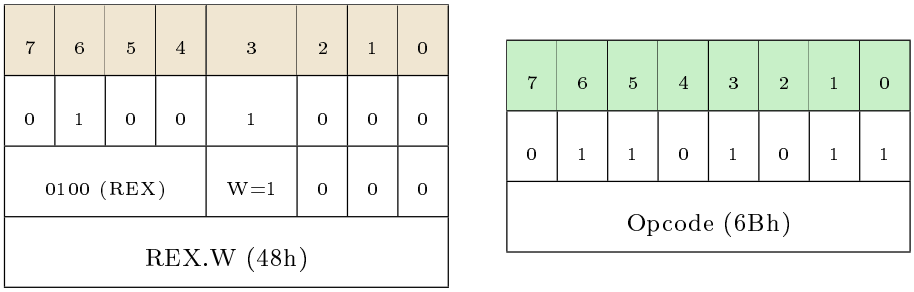

B8 - код mov

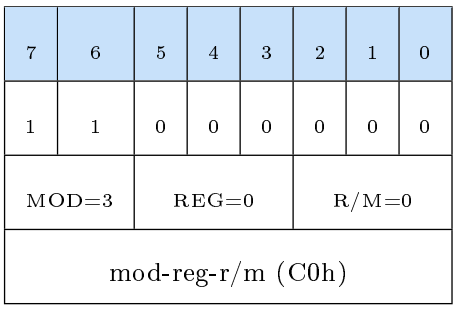
reg = 0 - eax

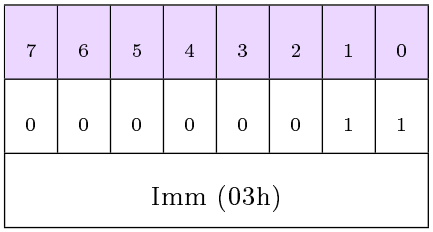
Immediate = 00000004h

непосредственная адресация

**00007FF630CD1741** **48 6B C0 03** `imul rax, rax, 3`






6B - код imul

MOD = 3 - регистровая адресация

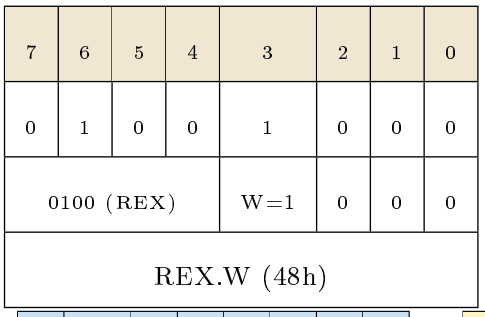
REG = 0 - eax

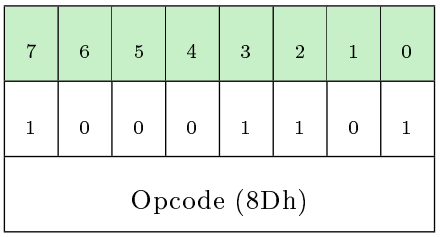
R/M = 0 - eax

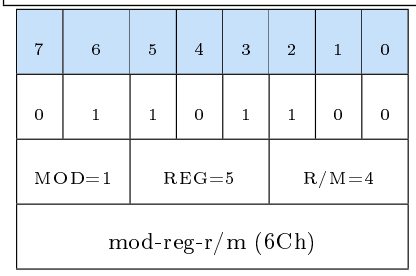
Immediate = 03h

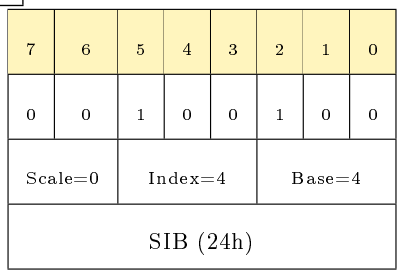
регистровая и непосредственная адресация

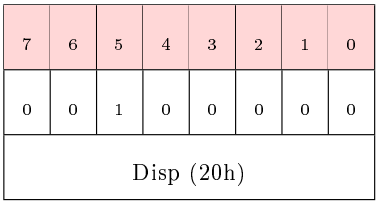
**00007FF630CD172A** **48 8D 6C 24 20** `lea rbp, [rsp+20h]`




8D - LEA

MOD = 1 - адресация памяти со смещением disp8

REG = 5 - rbp

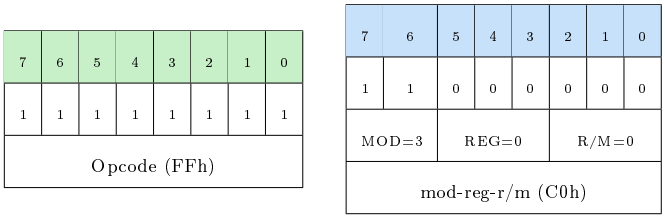
R/M = 4 - смотрим SIB

Scale = 0 (x1), Index = 4, Base = 4

Displacement = 20h (смещение +32 байта)

косвенная регистровая адресация со смещением

**00007FF630CD17D4** **FF C0** `inc eax`




FF - INC

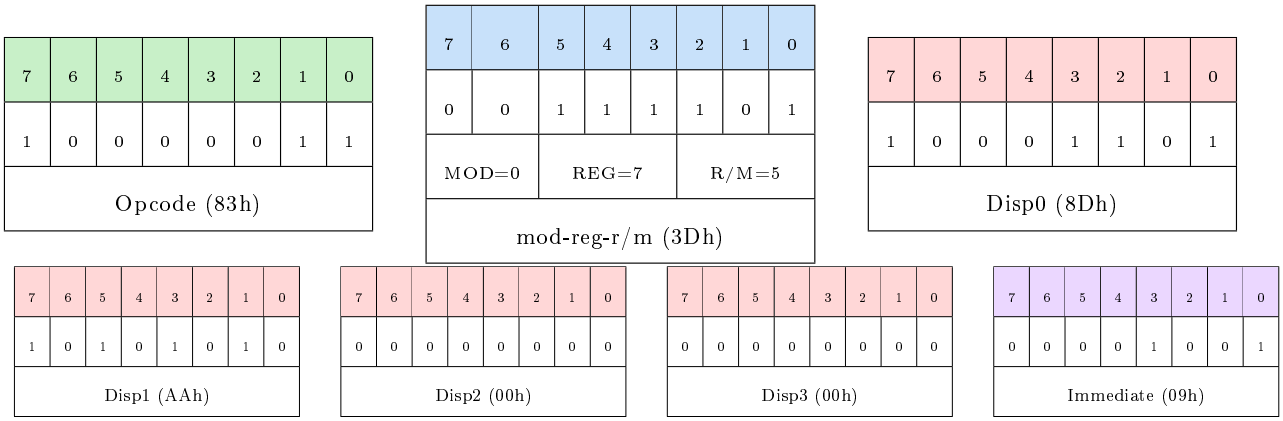
MOD = 3 - регистровая адресация

REG = 0

R/M = 0 - eax

регистровая адресация

**00007FF630CD17DC** **83 3D 8D AA 00 00 09** cmp dword ptr [i], 9




83 - CMP

MOD = 0 - displacement содержит адрес переменной в памяти

REG = 7 - CMP

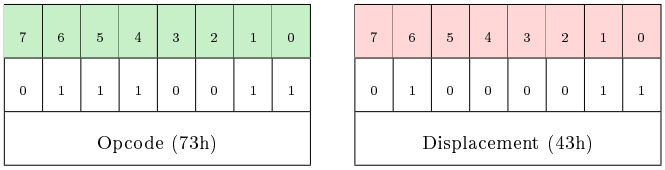
R/M = 5 ebx

Displacement = 0000AA8Dh

Immediate = 09h

прямая адресация памяти и непосредственная адресация

**00007FF630CD17E3** **73 43** jae main+108h (07FF630CD1828h)




73 cb - JAE

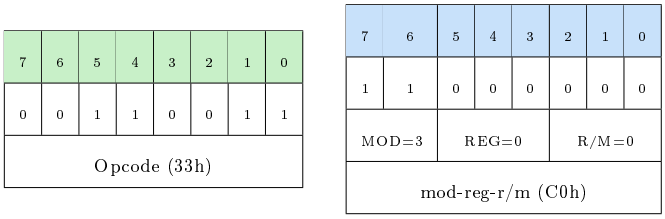
IP = 00007FF630CD17E5h (адрес следующей команды)

next_command_address = IP + 0x43 = 07FF630CD1828h, если соблюдается

условие ($i \geq 9$)

относительная адресация

**00007FF630CD1828** **33 C0** `xor eax, eax`




`33 - XOR`

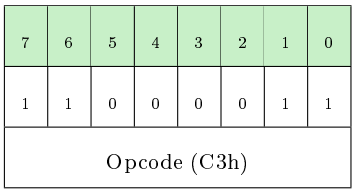
`MOD = 3 - регистровая адресация (R/M содержит номер регистра)`

`REG = 0 - eax`

`R/M = 0 - eax`

регистровая адресация

**00007FF630CD1833** **C3** `ret`




`C3 - RET`

неявная адресация

**5. Многокомпонентная адресация данных**

Многокомпонентная масштабно-индексная адресация использована в команде записи в массив в теле цикла:

`mov dword ptr [rdx+rcx*4], eax` (машинный код: `89 04 8A`)

Значения отдельных компонент исполнительного адреса:

- **База (Base):** регистр `rdx` — содержит базовый адрес начала массива `iMas` (`0x07FF630CDC240`);
- **Индекс (Index):** регистр `rcx` — содержит значение переменной `i` ($0 \leq i < 9$);
- **Масштаб (Scale):** коэффициент 4 — соответствует размеру типа `sizeof(in` `= 4` байта;

- **Смещение (Displacement):** отсутствует (равно 0).

Исполнительный адрес вычисляется аппаратно за один такт: $EA = rdx + rcx \cdot 4$.

### 6. Способы адресации в командах перехода

Во всех командах ветвления компилятор применил **относительную адресацию** (`rel8`): адрес перехода не хранится целиком, а задаётся знаковым смещением относительно счётчика команд следующей инструкции (Target = IP + offset):

- `EB 0E (jmp main+0BCh)`: безусловный короткий переход на проверку условия;
- `73 43 (jae main+108h)`: беззнаковый переход при выходе из цикла ($i \geq 9$);
- `72 1C (jb main+0EAh)`: беззнаковый переход в ветку `else` при $i < 4$ (по флагу $CF = 1$);
- `EB 1C (jmp main+106h)`: безусловный переход в обход ветки `else`;
- `EB A6 (jmp main+0CEh)`: безусловный переход на шаг цикла со смещением `0xA6` ($-90$).

### 7. Модификация программы: доступ через указатель и локальные переменные

В соответствии с шагом 10 методики доступ к массиву изменён на последовательное обращение через указатель `pMas` (листинг 3).

Листинг 3: Модифицированная программа с доступом через указатель

```
int iMas[10];
int *pI;
unsigned int i;

int main() {
    iMas[3] = 12;
    iMas[4] = 12345;
    iMas[5] = 12345678;
    iMas[6] = 0x123456789ABCDE;

    pI = &iMas[2];
    *pI = 0xAB12;
```

```
13
14      int *pMas = iMas;
15      for (i = 0; i < 9; i++, pMas++) {
16          if (i >= 4)
17              *pMas = 13 * i + 7;
18          else
19              *pMas = (i * 8) ^ 0xC;
20      }
21      return 0;
22  }
23
```

Дизассемблированный фрагмент модифицированного цикла:

Листинг 4: Дизассемблерный код цикла при работе через указатель

```
1  00007FF721F317C2 48 8D 05 77 AA 00 00 lea          rax,[iMas (07FF721F3C240h)
       ]           ; Получение адреса начала массива iMas
2  00007FF721F317C9 48 89 45 08          mov          qword ptr [pMas],rax
                   ; Сохранение адреса в локальный указатель pMas
3  15:      for (i = 0; i < 9; i++, pMas++) {
4       00007FF721F317CD C7 05 99 AA 00 00 00 00 00 00 mov          dword ptr [i
       (07FF721F3C270h)],0 ; Инициализация счетчика цикла: i = 0
5       00007FF721F317D7 EB 1A                jmp          main+0D3h (07
       FF721F317F3h)            ; Безусловный переход к проверке условия цикла
6       00007FF721F317D9 8B 05 91 AA 00 00    mov          eax,dword ptr [i (07
       FF721F3C270h)]    ; Загрузка значения i в eax
7       00007FF721F317DF FF C0                inc          eax
                       ; Инкремент счетчика (i++)
8       00007FF721F317E1 89 05 89 AA 00 00    mov          dword ptr [i (07
       FF721F3C270h)],eax    ; Сохранение обновленного i
9       00007FF721F317E7 48 8B 45 08          mov          rax,qword ptr [pMas]
                       ; Загрузка адреса текущего элемента из pMas
10      00007FF721F317EB 48 83 C0 04          add          rax,4
                       ; Инкремент указателя на 4 байта (pMas++)
11      00007FF721F317EF 48 89 45 08          mov          qword ptr [pMas],rax
                       ; Сохранение нового адреса в pMas
12      00007FF721F317F3 83 3D 76 AA 00 00 09 cmp          dword ptr [i (07
       FF721F3C270h)],9      ; Сравнение i с 9
13      00007FF721F317FA 73 2F                jae          main+10Bh (07
       FF721F3182Bh)            ; Если i >= 9, выход из цикла беззнаковый( переход)
14      16:          if (i >= 4)
15      00007FF721F317FC 83 3D 6D AA 00 00 04 cmp          dword ptr [i (07
       FF721F3C270h)],4      ; Сравнение i с 4
16      00007FF721F31803 72 12                jb           main+0F7h (07
       FF721F31817h)            ; Если i < 4, переход в ветку else (jb, CF=1)
17      17:              *pMas = 13 * i + 7;
```

```
00007FF721F31805 6B 05 64 AA 00 00 0D imul        eax,dword ptr [i (07FF721F3C270h)],0Dh ; eax = i * 13 (0x0D)
00007FF721F3180C 83 C0 07             add         eax,7
                 ; eax = 13 * i + 7
00007FF721F3180F 48 8B 4D 08          mov         rcx,qword ptr [pMas]
                 ; Загрузка адреса элемента из указателя pMas в rcx
00007FF721F31813 89 01                mov         dword ptr [rcx],eax
                 ; Косвеннорегистровая- запись [rcx] код( 89 01)
00007FF721F31815 EB 12                jmp         main+109h (07FF721F31829h)            ; Обход ветки else
18:          else
19:              *pMas = (i * 8) ^ 0xC;
00007FF721F31817 8B 05 53 AA 00 00    mov         eax,dword ptr [i (07FF721F3C270h)]    ; Загрузка i в eax
00007FF721F3181D C1 E0 03             shl         eax,3
                 ; Умножение на 8 сдвигом влево на 3 бита
00007FF721F31820 83 F0 0C             xor         eax,0Ch
                 ; Побитовое исключающее ИЛИ с константой 12
00007FF721F31823 48 8B 4D 08          mov         rcx,qword ptr [pMas]
                 ; Загрузка адреса элемента из указателя pMas в rcx
00007FF721F31827 89 01                mov         dword ptr [rcx],eax
                 ; Косвеннорегистровая- запись [rcx] код( 89 01)
```

**Сравнение способов адресации:**

1. **Индекс против указателя:** При доступе по индексу процессор на каждом шаге вычисляет формулу $rdx+rcx\cdot4$ командой `mov [rdx+rcx*4], eax` (машинный код `89 04 8A`, 3 байта, содержит SIB-байт). При доступе через указатель запись выполняется короткой командой косвенно-регистровой адресации `mov [rcx], eax` (код `89 01`, 2 байта, без SIB-байта), а шаг к следующему элементу сводится к простому сложению `add rax, 4`.
2. **Глобальное против локального размещения:** Глобальные переменные размещены в статической памяти и адресуются по фиксированным смещениям. Локальные переменные размещаются в стеке и адресуются со смещением относительно регистра базы кадра: `[rbp-offset]`.

**Выводы:**

В ходе лабораторной работы изучена система команд и способы адресации процессоров архитектуры x86. Проанализировано размещение глобальных данных в памяти, составлена карта памяти. Установлено, что при об-

ращении к элементам массива по индексу используется многокомпонентная масштабно-индексная адресация с масштабом 4 (размер типа `int`). Замена индексации на указатель упрощает команду записи до короткой косвенно-регистровой (`mov [rcx], eax`) без вычисления адреса на каждой итерации. Проверка условий беззнаковых переменных (`unsigned`) транслируется в беззнаковые переходы (`jb`, `jae`), а умножение на степень двойки ($i \cdot 8$) компилятор эффективно заменяет побитовым сдвигом влево (`shl eax, 3`).